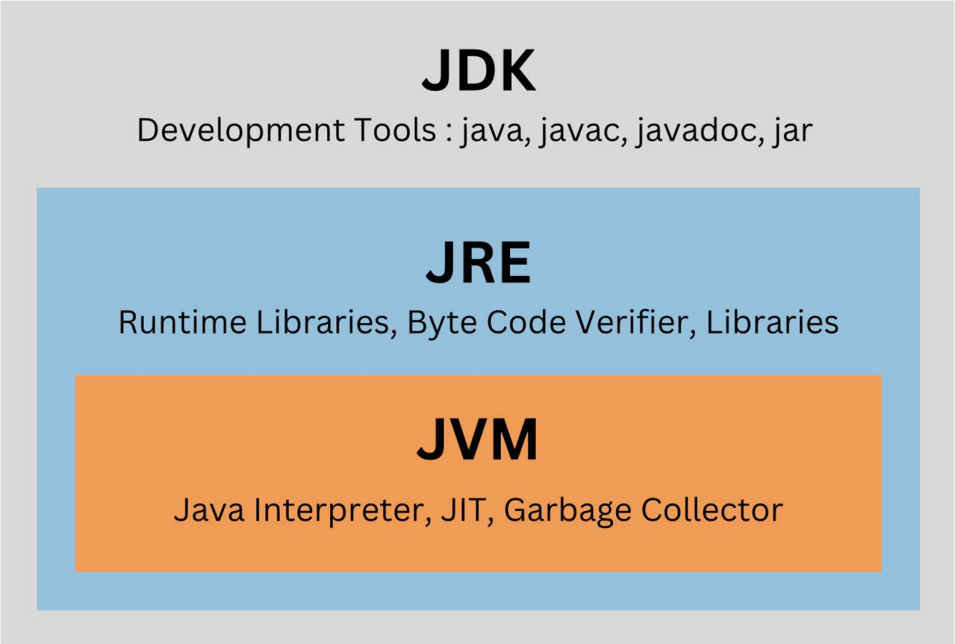
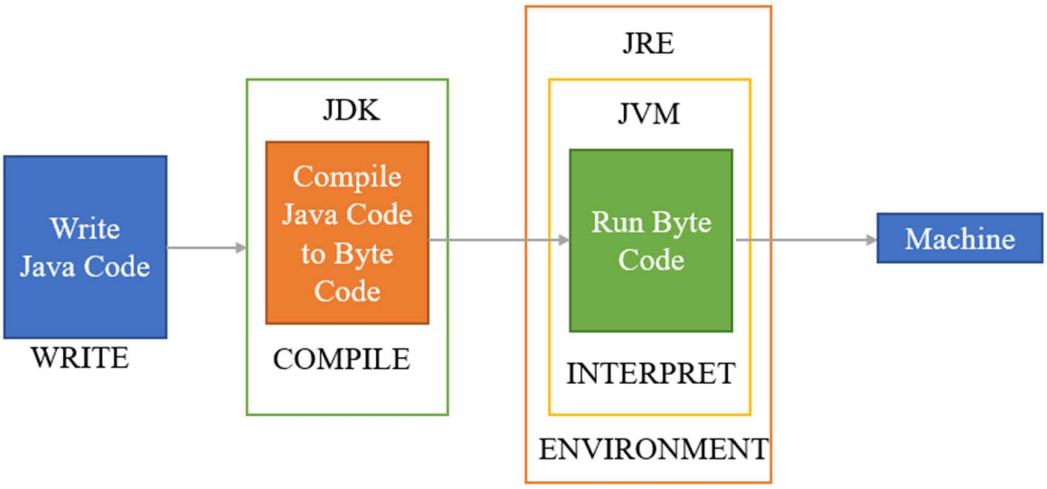
JDK JRE JVM

Architecture Diagram :-



How to perform JDK, JRE, JVM? :-



Difference Between JDK, JRE, JVM :-

JDK vs JRE vs JVM

Aspect	JDK	JRE	JVM
Purpose	Used to develop Java applications	Used to run Java applications	Executes Java bytecode
Platform Dependency	Platform-dependent (OS specific)	Platform-dependent (OS specific)	JVM is OS-specific, but bytecode is platform-independent
Includes	JRE + Development tools (javac, debugger, etc.)	JVM + Libraries (e.g., rt.jar)	ClassLoader, JIT Compiler, Garbage Collector
Use Case	Writing and compiling Java code	Running a Java application on a system	Convert bytecode into native machine code

Difference between Compiler and Interpreter:-

Feature	Compiler	Interpreter
Translation	Entire program at once, before execution	Line by line, during execution
Output	Standalone executable file	No separate executable; direct execution
Speed	Faster execution time once compiled	Slower execution time
Error Detection	All errors are reported after compilation	Errors are reported as they occur during runtime
Execution	Runs independently of source code	Requires source code and interpreter at runtime

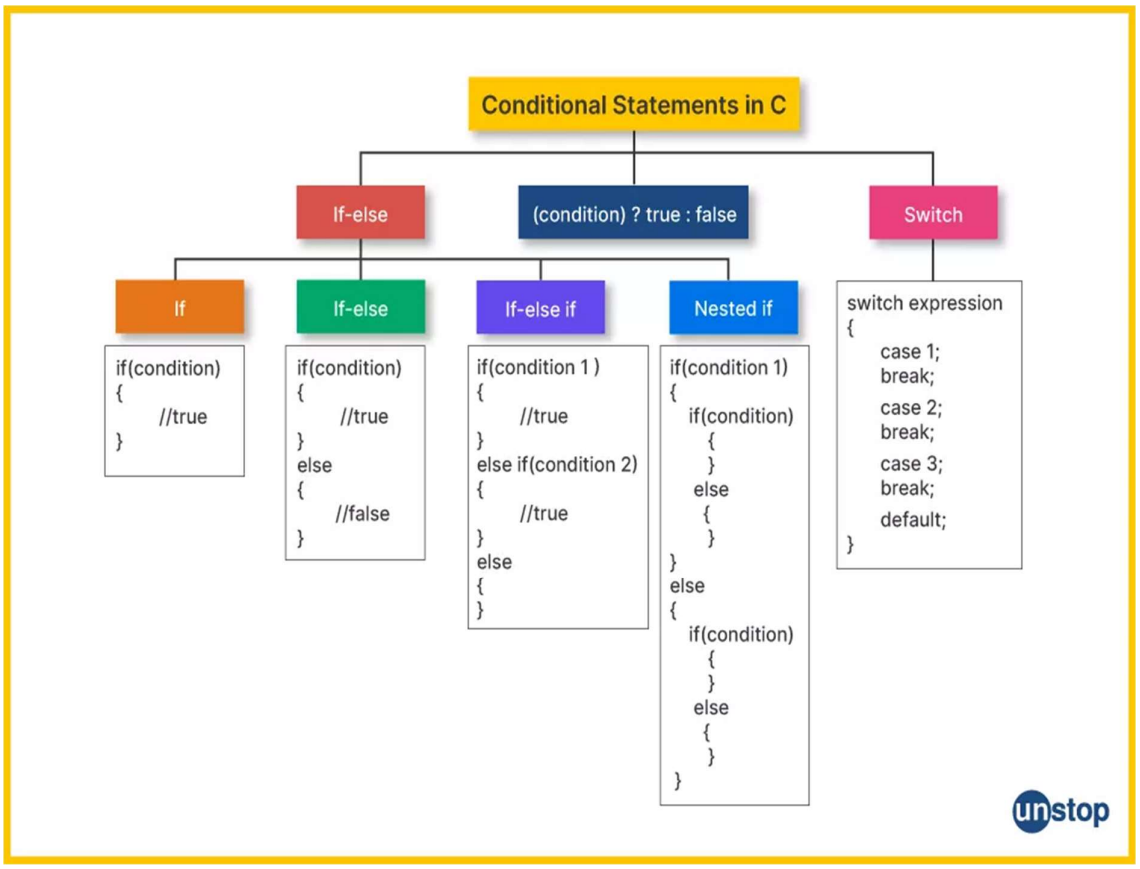
Conditional Statement

- **if statement :-**
  - Every if statement needs a condition that results in true or false.

- **if-else statement :-**
  - The else statement lets you run a block of code when the condition in the if statement is false.

- **else-if ladder statement :-**
  - Use the else if statement to specify a new condition if the first condition is false.

- **nested if-else statement :-**
  - You can also place an if statement inside another if. This is called a nested if statement.



Implementation

if statement :-

Input :-

```java
package basicconcepts;

public class ifelsestatement {
    public static void main(String[] args) {
        int num=90;

        if(num==90)
        {
            System.out.println("Equals");
        }

    }
}
```

Output :-

```
Equals
```

if-else statement :-

Input :-

```java
package basicconcepts;

public class ifelsestatement {
    public static void main(String[] args) {
        int num=90;
        if(num!=90)
        {
            System.out.println("Equals");
        }
        else
        {
            System.out.println("Not Equal");
        }
    }
}

```

Output :-

```
Not Equal
```

else-if ladder statement :-

Input :-

```java
package basicconcepts;

public class elseifladder {
    public static void main(String[] args) {

        int num=10;

        if(num==20)
        {
            System.out.println("Condition 1 is true");
        }
        else if(num>=55)
        {
            System.out.println("Condition 2 is True");
        }
        else if(num<9)
        {
            System.out.println("Condition 3 is True");
        }
        else
        {
            System.out.println("All Condition are False");
        }

    }

}
```

Output :-

```
All Condition are False
```

nested if-else statement :-

Input :-

```java
package basicconcepts;

public class nestedifelse {

    public static void main(String[] args) {

        boolean iselectricity=true;
        boolean isswitchon=true;

        if(iselectricity==true)
        {
            if(isswitchon==true)
            {
                System.out.println("lights on!!");
            }
            else
            {
                System.out.println("lights off!!");
            }
        }
        else
        {
            System.out.println("No Electricity");
        }
    }
}
```

Output :-

```
lights on!!
```